Machine Learning

Módulo 1

Transformación de Datos

Transformación de datos

Es el proceso que más tiempo lleva en un flujo de Data Science y resulta muy importante no perder el objetivo de **por qué lo hacemos**.

Por un lado, los modelos de Machine Learning que usemos, que van a "aprender" de nuestros datos, sólo entienden de números. Esto quiere decir que, si en un dataset hay una columna de género con dos valores, ‘femenino’ y ‘masculino’, tendremos que llevarlo a una representación que los modelos entiendan. En este caso, asignamos un ‘0’ y un ‘1’ a cada uno de los valores y problema resuelto.

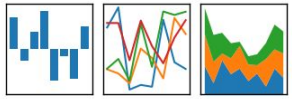
En el fondo, hay una hipótesis implícita: el género es un **campo muy útil para hacer la tarea de predicción o clasificación** que requerimos, por lo que queremos mantener esta información y hacerla comprensible para nuestros modelos. La pregunta que queremos responder nos va a indicar cómo tenemos que trabajar con nuestro dataset.



Transformación de datos

pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



Variables Numéricas

En general, las variables numéricas podrían ser usadas sin demasiado preprocesamiento, ya que ya están en un formato que los modelos entienden. Sin embargo, muchas veces no es ese el caso.

Discretización y binning:
La función *cut()* transforma la columna Age en rangos (bins) convirtiendo así el dato de edad en una variable discreta.

```
bins = [0,3,12,18,60,100]
labels = ['bebe', 'niño', 'adolescente', 'adulto', 'anciano']
```

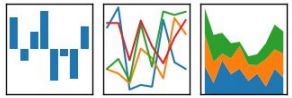
```
cats = pd.cut(df.Age, bins, labels = labels)
df['Age_labels'] = cats
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_labels
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	adulto
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	adulto
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	adulto
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	adulto
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S	adulto

Transformación de datos

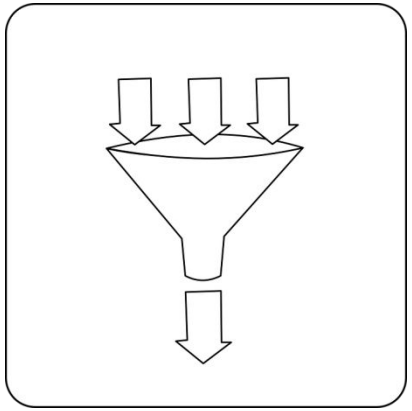
pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



Variables Numéricas

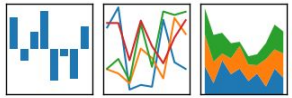
- Reescalar, esto es útil sobre todo cuando queremos llevar los datos a una distribución de probabilidad con la que podamos trabajar mejor, como en el caso que vimos de llevar el dato a su logaritmo base 10.
- Combinar con otras variables



Transformación de datos

pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



Variables Nominales

Este es uno de los tipos de **encoding** más comunes que vamos a tener que hacer.

En el ejemplo, queremos llevar al género, los valores male y female, a 0 y 1. Lo importante es no perder cuál es cuál. Esto, en Pandas, lo podemos hacer con la función *map()*.

Este tipo de encoding se denomina **Label_enconding**

```
diccionario = {'male': 0, 'female': 1}

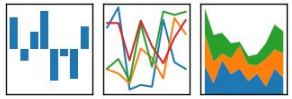
df['Sex_Map'] = df.Sex.map(diccionario)
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_labels	Sex_Map
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	adulto	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	adulto	1
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	adulto	1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	adulto	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S	adulto	0

Transformación de datos

pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



Variables Dummies

Una variable dummy toma como valor 0 o 1 para indicar la presencia o ausencia de algún atributo categórico.

La función *get_dummies()* hace automáticamente esto en un dataframe sobre las columnas indicadas. Este tipo de encoding se denomina **One-hot-encoding**

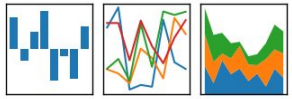
```
df = pd.concat([df, pd.get_dummies(df['Sex'])], axis=1)
df.head()
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_labels	Sex_Map	female	male
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	adulto	0	0	1
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	adulto	1	1	0
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	adulto	1	1	0
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	adulto	1	1	0

Transformación de datos

pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



Valores Faltantes

- Con la función *isna()* podemos obtener una matriz que oficie de 'máscara' y determine los valores nulos.
- Con la función *dropna()* podemos eliminar filas con valores nulos.

```
usuarios_df = pd.DataFrame({'edad': [np.nan, 20, 41, 20, 35, 42, 24, np.nan],
                            'clics_por_hora': [156, np.nan, 210, 210, np.nan, 130, 170,220]})
usuarios_df
```

	edad	clics_por_hora
0	NaN	156.0
1	20.0	NaN
2	41.0	210.0
3	20.0	210.0
4	35.0	NaN
5	42.0	130.0
6	24.0	170.0
7	NaN	220.0

- Con la función *fillna()* podemos rellenar los valores nulos bajo un cierto criterio.

```
usuarios_df['edad'].fillna(usuarios_df.edad.mean(), inplace=True)

usuarios_df['clics_por_hora'].fillna(usuarios_df.clics_por_hora.min(), inplace=True)

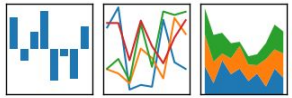
usuarios_df
```

	edad	clics_por_hora
0	30.333333	156.0
1	20.000000	130.0
2	41.000000	210.0
3	20.000000	210.0
4	35.000000	130.0
5	42.000000	130.0
6	24.000000	170.0
7	30.333333	220.0

Transformación de datos

pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



Valores Duplicados

Con la función *drop_duplicates()* nos podemos deshacer de filas duplicadas, incluso eligiendo por ciertos campos **clave** (un campo clave identifica al registro de manera unívoca y pueden ser uno o la combinación de varios).

```
ciudades_df = pd.DataFrame({'ciudad': ['Cordoba','Rosario','Gualeguay','Gualeguay','Ushuaia'],
                            'provincia': ['Cordoba','Santa Fe','Entre Rios','Entre Rios','Tierra del Fuego'],
                            'Region': ['Centro','Centro','Mesopotamia','Mesopotamia','Patagonia']})
ciudades_df
```

	ciudad	provincia	Region
0	Cordoba	Cordoba	Centro
1	Rosario	Santa Fe	Centro
2	Gualeguay	Entre Rios	Mesopotamia
3	Gualeguay	Entre Rios	Mesopotamia
4	Ushuaia	Tierra del Fuego	Patagonia

```
ciudades_df.drop_duplicates(subset=['ciudad'])
```

	ciudad	provincia	Region
0	Cordoba	Cordoba	Centro
1	Rosario	Santa Fe	Centro
2	Gualeguay	Entre Rios	Mesopotamia
4	Ushuaia	Tierra del Fuego	Patagonia

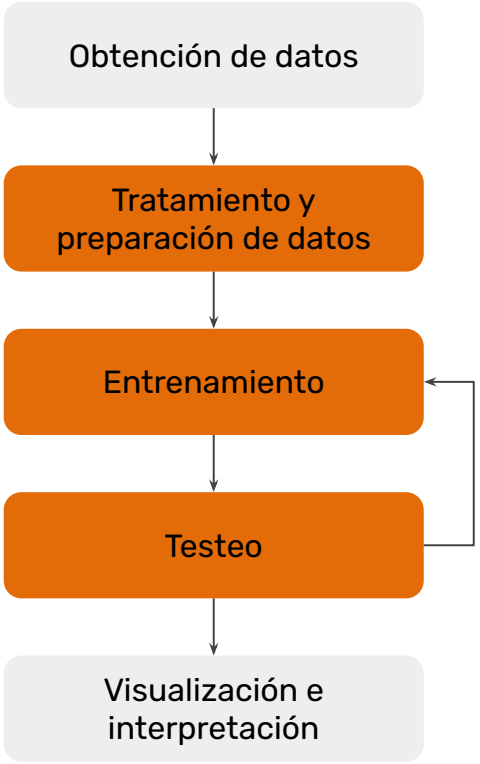
Scikit-Learn

Scikit-Learn es la librería base para Machine Learning en Python. Se puede usar para:

- Procesamiento de los datos
- Modelos de Clasificación y Regresión
- Métricas de **Evaluación de algoritmos**

Adecuado para seguir un **Pipeline:** Es un objeto en el que es posible encapsular todo el proceso de los datos.

Documentación: *https://scikit-learn.org/stable/index.html*



Scikit-Learn

Vamos a encontrar que Scikit-Learn trabaja con Clases e implementa de manera uniforme los atributos y métodos de sus objetos:

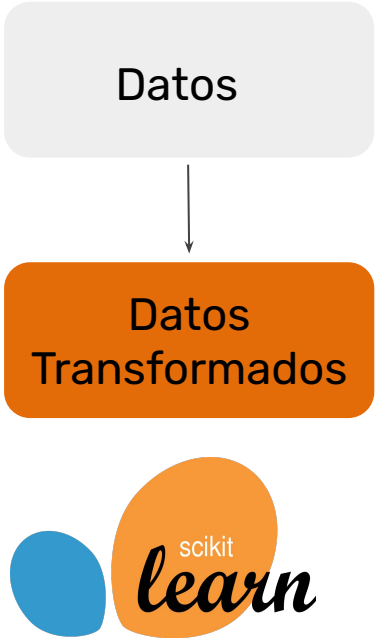
- **Estimadores:** Todos tienen el método **fit**()
- **Predictores:** Todos tienen el método **predict**()
- **Transformadores:** Todos tienen el método **transform**()
- **Modelos:** Todos tienen el método **score**()



Scikit-Learn

En general, los pasos a seguir con una biblioteca de Scikit-Learn serán:

1. Crear el objeto con los parámetros correspondientes.
   objeto = Clase(parametros)
2. Implementar el método fit() que aprende de los datos.
   objeto.fit(Datos)
3. Implementar el método transform() que transforma los datos.
   objeto.transform(Datos)



Scikit-Learn

Transformación de datos

Las siguientes clases son las herramientas disponibles para procesar datos:

- **SimpleImputer:** Rellena valores faltantes.
- **OneHotEncoder:** Pasa de variables categóricas a dummies.
- **LabelEncoder:** Pasa variables categóricas a valores numéricos.
- **KBinsDiscretizer:** Para discretización y binning, la principal diferencia con Pandas es que Scikit-Learn decide los límites de los bines de acuerdo a una estrategia que le pasemos de parámetro.
- **SelectKBest:** Selecciona atributos del dataset en base a diferentes criterios de evaluación. Puede servir como respaldo o referencia del análisis que se está realizando sobre los datos.



Scikit-Learn - Transformación de datos

SimpleImputer

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='mean')
```

```
clics = usuarios_df.clics_por_hora.values
imp.fit(clics.reshape(-1,1))
print(imp.statistics_)
```

```
[182.66666667]
```

```
clics_imputed = imp.transform(clics.reshape(-1,1))
```

```
usuarios_df['clics_por_hora_imputed'] = clics_imputed
usuarios_df
```

	edad	clics_por_hora	explorador	clics_por_hora_imputed
0	32	156.0	Chrome	156.000000
1	20	NaN	Opera	182.666667
2	41	210.0	Firefox	210.000000
3	20	210.0	Chrome	210.000000
4	35	NaN	Explorer	182.666667
5	42	130.0	Firefox	130.000000
6	24	170.0	Chrome	170.000000
7	28	220.0	Chrome	220.000000



Scikit-Learn - Transformación de datos

```
from sklearn.preprocessing import OneHotEncoder
onehot_encoder = OneHotEncoder(sparse = False)
```

```
exploradores = usuarios_df.explorador.values.reshape(-1,1)
```

```
onehot_encoder.fit(exploradores)
```

```
OneHotEncoder(categories='auto', drop=None, dtype=<class 'numpy.float64'>,
              handle_unknown='error', sparse=False)
```

```
onehot_encoder.categories_
```

```
[array(['Chrome', 'Explorer', 'Firefox', 'Opera'], dtype=object)]
```

```
explorador_encoded = onehot_encoder.transform(exploradores)
print(explorador_encoded)
```

```
[[1. 0. 0. 0.]
 [0. 0. 0. 1.]
 [0. 0. 1. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]]
```

OneHotEncoder:
Notar que con N instancias, son necesarias solo N-1 nuevas columnas.

```
usuarios_df['chrome_encoded'] = explorador_encoded[:,0]
usuarios_df['explorer_encoded'] = explorador_encoded[:,1]
usuarios_df['firefox_encoded'] = explorador_encoded[:,2]
usuarios_df['opera_encoded'] = explorador_encoded[:,3]
usuarios_df
```

	edad	clics_por_hora	explorador	clics_por_hora_imputed	chrome_encoded	explorer_encoded	firefox_encoded	opera_encoded
0	32	156.0	Chrome	156.000000	1.0	0.0	0.0	0.0
1	20	NaN	Opera	182.666667	0.0	0.0	0.0	1.0
2	41	210.0	Firefox	210.000000	0.0	0.0	1.0	0.0
3	20	210.0	Chrome	210.000000	1.0	0.0	0.0	0.0
4	35	NaN	Explorer	182.666667	0.0	1.0	0.0	0.0
5	42	130.0	Firefox	130.000000	0.0	0.0	1.0	0.0
6	24	170.0	Chrome	170.000000	1.0	0.0	0.0	0.0
7	28	220.0	Chrome	220.000000	1.0	0.0	0.0	0.0

Scikit-Learn - Transformación de datos

LabelEncoder

```
from sklearn import preprocessing
le = preprocessing.LabelEncoder()
```

```
le.fit(usuarios_df['explorador'])
```

```
LabelEncoder()
```

```
usuarios_df['explorador_le'] = le.transform(usuarios_df['explorador'])
```

```
usuarios_df
```

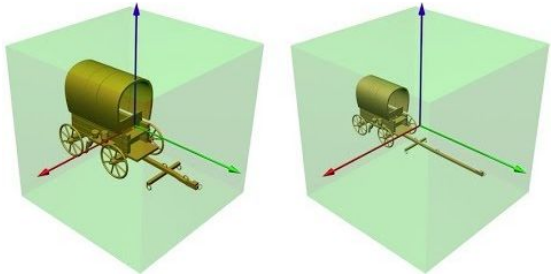
	edad	clics_por_hora	explorador	clics_por_hora_imputed	explorador_le
0	32	156.0	Chrome	156.000000	0
1	20	NaN	Opera	182.666667	3
2	41	210.0	Firefox	210.000000	2
3	20	210.0	Chrome	210.000000	0
4	35	NaN	Explorer	182.666667	1
5	42	130.0	Firefox	130.000000	2
6	24	170.0	Chrome	170.000000	0
7	28	220.0	Chrome	220.000000	0



Scikit-Learn

Reescalar datos

Muchos algoritmos funcionan mejor normalizando sus variables de entrada. Lo que en este caso significa comprimir o extender los valores de la variable para que estén en un rango definido. Sin embargo, una mala aplicación de la normalización o una elección descuidada del método de normalización puede arruinar los datos y, con ello, el análisis.



Dos de los criterios más conocidos para reescalar son:

- MinMax Scaler
- Standard Scaler

Scikit-Learn - Reescalar datos

MinMax Scaler:

- Las entradas se normalizan entre dos **límites definidos**:

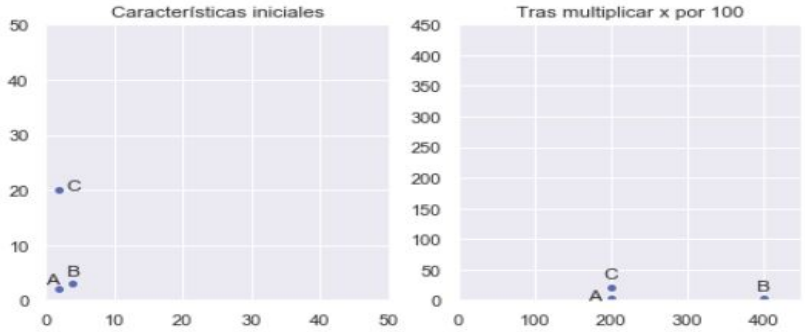
$$X_{normalize} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

d

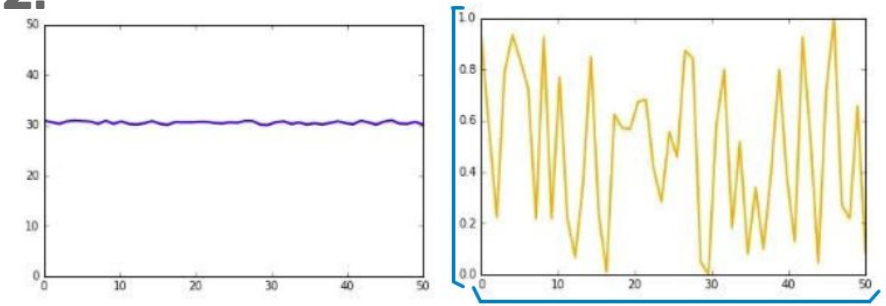
- Tener en cuenta que si se reescala un atributo, quizás sea conveniente reescalar otro, debido a que estamos rompiendo la **proporcionalidad de los datos**:

  En 1 originalmente, A estaba más cerca de B, al multiplicar por 100, quedó más cerca de C. En 2 el ruido de la señal se hizo más notorio.

1.



2.



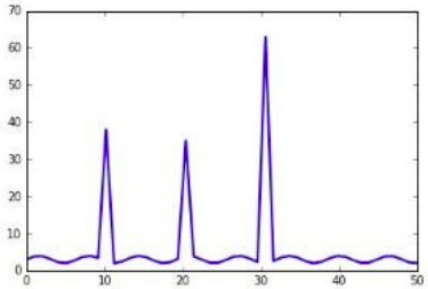
Scikit-Learn - Reescalar datos

Standard Scaler:

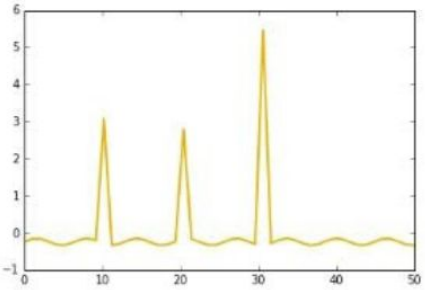
- A cada dato se le resta la media de la variable y se le divide por la desviación típica:

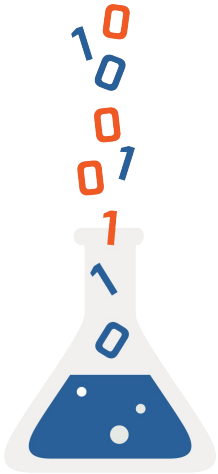
$$X_{normalized} = \frac{X - X_{mean}}{X_{stddev}}$$

- Si bien puede resultar conveniente en datos que no tienen distribución de probabilidad Gaussiana o Normal debido a que se puede trabajar mejor bajo ese esquema, tanto la media como la desviación típica son muy sensibles a **outliers**.



Ahora hay valores negativos, cuando antes no. Y los valores pico y valle han quedado muy atenuados debido a las anomalías.





Lab Time!

¡Muchas gracias!

¡Sigamos trabajando!